LeakLens

Water Leak Localization System

How the Models Evolved, Step by Step

This document explains, in simple language, every version of the detection and localization method that existed before the final one, why each was replaced, and what each scored. It also lists what we did *not* try. All data are simulated.

Reading the numbers. Early numbers were measured on all scenarios (no separate test set), so they are only a guide to the *direction* of improvement. The final numbers come from the 80/20 hold-out test (139 unseen scenarios, 60 scored leaks) and are the only ones to quote as results. Where the same method appears in both, both are shown.

1 The timeline in one table

Version	What it was	Main problem or result
V0	Original Semester 1 prototype	Fixed 0.5 m alarm and a pressure + flow score. Tested on 2 large leaks ("100%"). Cannot name causes; many false alarms.
V1	First new localizer	Pressure and flow added in raw units. Pressure dominated. Top-1 84.9%.
V2	Scaled signals + neighbour smoothing	Both signals scaled 0 to 1; unaccounted flow made the main signal. Top-1 95.9%.
V3	Final localizer	Same as V2 without smoothing. Top-1 97.6% (all scenarios), 96.7% (unseen test).
Detector	Fixed → learned threshold → + quiet check + cause model	False alarms 41.7% → 0%.
Cause model	Decision tree vs random forest	Forest chosen: 82.7% vs 69.1% on unseen test.
Testing	3-fold cross-validation → 80/20 hold-out	Cross-validation was too optimistic.

2 V0: the original prototype

This was written before our new work (code in commit `d9c2911`; described in `SEMESTER_1_DOCUMENTATION.md`). It has two parts, neither of which learns anything.

2.1 Step by step: how V0 worked

1. **Compare with the normal day.** For every node and hour: residual = normal pressure − observed pressure.
2. **Detect.** If any node's residual is above **0.5 m** at any hour, say "anomaly".
3. **Net flow.** For each node add up the change in flow of the pipes that touch it (flow in minus flow out). Keep the largest positive value, called the divergence.
4. **Score each node:**
$$\text{score}_i = \max_t r_i(t) \; + \; 50 \times \max\{0,\ \text{divergence}_i\}.$$
5. **Rank.** Highest score first. First node is the guess.

2.2 Why it was weak

- **0.5 m is a guess.** Sensors are never perfect, so normal noise crosses 0.5 m on some days. On our test days it raised a false alarm **41.7%** of the time.
- **It cannot tell a leak from anything else.** A pump stopping, a tank low or demand rising also lowers pressure by more than 0.5 m. The screen said "cause not classified".
- **The flow part could not separate a leak from customer use.** It counted *any* extra flow leaving a node, including water customers legitimately used. A demand surge therefore looked like a leak.
- **The 50 was arbitrary,** and pressure and flow were added in different units.
- **It was only tested on two huge leaks,** which are easy. The "100%" result said little about general performance.

2.3 How V0 scores on our test

	On all scenarios	On the unseen test
Exact leak node first (Top-1)	56.4%	58.3%
Leak in first five (Top-5)	81.1%	71.7%
Average miss (pipes)	3.81	3.35
False alarms on normal days	37.5%	41.7%

We kept V0's ranking in the code as `LegacyLocalizer` so it can always be compared.

3 V1: unaccounted flow, first attempt

Idea. Instead of raw divergence, subtract what customers use. What remains is *unaccounted flow*, the real leak flow:

$$a_i = \big[\text{change in net inflow}\big] - \big[\text{change in metered demand}\big].$$

We also scaled pressure and flow by the learned noise level, so "1" means "at the noise limit".

First scoring attempt. We simply added the two signals: score = d_i + (unaccounted flow), both in noise units.

What went wrong. Top-1 was only **84.9%**. We looked at the misses and found why: in Net2, one leak at node 2 gave unaccounted flow 11.4 (noise units) at node 2, but node 1 showed a *pressure drop* of 128. Pressure drops are large and spread to many nodes, so the pressure term drowned the flow signal and node 1 won. Adding signals measured in different scales does not work.

4 V2: scale each signal, flow first

Fix. Divide each signal by its own largest value in the network, so each runs from 0 to 1, and let flow lead:

$$\text{score}_i = A_i + 0.3\,D_i, \qquad A_i = \frac{a_i}{\max_j a_j}, \quad D_i = \frac{d_i}{\max_j d_j}.$$

We also tried **neighbour smoothing**: add 0.3 times the average score of the neighbouring nodes, on the idea that pressure drops spread along pipes. **Result:** Top-1 jumped from 84.9% to **95.9%**.

5 V3: the final localizer

Testing the variants side by side showed smoothing did not help: on the all-scenarios run it *lowered* Top-1 (97.6% without, 95.9% with); on the unseen test both gave the same 96.7%. Because it added complexity and no benefit, we removed it. This is the final method: **score** = $A + 0.3D$**, no smoothing**.

Variant (unseen test, 60 leaks)	Top-1	Top-5	Avg. miss (pipes)
V0: pressure + 50 × flow	58.3%	71.7%	3.35
Final: $A + 0.3D$	**96.7%**	**98.3%**	**0.13**
Final + smoothing	96.7%	98.3%	0.13
Final, without demand meters	93.3%	98.3%	0.72
Pressure only	60.0%	70.0%	3.05
Pressure only + smoothing	50.0%	63.3%	3.18

Two lessons: unaccounted flow is what makes localization work (58.3% → 96.7%), and pressure alone is weak (60.0%) because pressure drops spread to neighbouring nodes.

6 The detector's evolution

1. **Fixed 0.5 m rule (V0).** Catches most problems but ignores noise: 41.7% false alarms.
2. **Learned threshold.** We measured the sensor noise from each network's normal runs and set the threshold at four times its spread (about 0.5–0.6 m), and required the drop to last two hours. False alarms fell to **0%**, but it missed more real problems (recall 77.2%) because it only looked at pressure.
3. **Quiet check + cause model.** We added the flow and demand signals and a classifier. A bug found on the way: a perfectly noise-free normal day was misread as a "blocked pipe" because the model had never seen perfectly quiet data. We fixed it by adding a rule: if every signal is within noise, the answer is normal.

Detector (unseen test)	Precision	Recall	F_1	False alarms
Fixed 0.5 m (V0)	95.7%	87.4%	91.4%	41.7%
Learned pressure threshold	100.0%	77.2%	87.1%	0.0%
Full system (final)	100.0%	93.7%	96.7%	0.0%

7 The cause classifier: two models compared

Two scikit-learn models were trained on the same 14 numbers and the same 478 training scenarios:

- **Decision tree, depth 4.** A short chain of yes/no questions. Easy to read; it is saved in `decision_rules.txt`.
- **Random forest, 300 trees.** Many trees, each trained on a random sample, that vote. Harder to read but more accurate.

Model (139 unseen test scenarios)	Accuracy	Macro-F_1
Decision tree (depth 4)	69.1%	51.3%
Random forest (300 trees), final	**82.7%**	**76.4%**
Random forest without demand meters	74.1%	65.4%

The forest was chosen. The tree is kept only to show readable rules. No settings were tuned for either.

8 How we tested: two methods

1. **3-fold cross-validation on all scenarios (replaced).** Split the scenarios into three groups; train on two, test on one; repeat. It gave a cause accuracy of 89.8% for the forest. But the same leak node at several sizes could land in both training and test groups, so the test was partly a memory check: too optimistic.
2. **80/20 hold-out (final).** Learn from 478 scenarios, test on 139 never used for learning. Near-copies stay together. The same forest scores **82.7%**. This is the honest figure.

9 What we did *not* try

Only the models above were built. We did **not** test any of the following, so we cannot say how they would compare:

- Support vector machines, gradient boosting, k-nearest neighbours, logistic regression.
- Neural networks, convolutional autoencoders with a one-class SVM (as in one of the papers we reviewed), or graph neural networks.
- Any tuning of settings: number of trees, tree depth, the 0.3 weight, or the quiet-check cut-offs. The 0.3 weight and cut-offs were chosen while looking at all scenarios, before the split, and not re-tuned.

A simple next step is to run several standard models on the same train/test split and report them in one table.

10 One-paragraph summary

We began with a fixed pressure alarm and a score that mixed pressure and flow (V0). Replacing raw flow with *unaccounted flow* was the key idea (V1); scaling both signals to a common 0–1 range and letting unaccounted flow lead fixed the ranking (V2); removing the unhelpful smoothing gave the final localizer (V3). A noise-aware detector removed the false alarms, a quiet check fixed a normal-day bug, and a random forest (better than a small tree) names the kind of event. Finally, an 80/20 hold-out test replaced cross-validation, so the final numbers are honest.